
BENCHMARKING GENERAL MOBILE ASSISTANTS IN CHALLENGING REAL-WORLD SCENARIOS

Yiqi Zhu¹, Feiyu Gao³, Jiaying Fan³, Jiahui Zeng³, Minggang Wu³

Chenliang Li³, Haiyang Xu³, Peng Li^{2,†}, Ming Yan^{3,†}, Yang Liu^{1,2,†}

¹Dept. of Comp. Sci. & Tech., Institute for AI, Tsinghua University, Beijing, China

²Institute for AI Industry Research (AIR), Tsinghua University, Beijing, China

³Alibaba Token Hub, Alibaba Group

zhuyq0218@gmail.com, lipeng@air.tsinghua.edu.cn

ym119608@alibaba-inc.com, liuyang2011@tsinghua.edu.cn

ABSTRACT

Graphical user interfaces have emerged as a primary environment for evaluating autonomous AI agents on multimodal interactive tasks. Existing benchmarks such as AndroidWorld and MobileWorld have established strong foundations for evaluating mobile agents in emulator-based Android tasks, but they remain limited in two respects. First, their application coverage does not yet fully capture the vast diversity of Android device ecosystems. Second, there is room for their task designs to be extended to encompass the nuanced complexity of realistic user requirements. To further propel the evaluation of mobile agents, we present GMA, a benchmark for assessing general mobile assistants in challenging real-world scenarios. To improve application ecosystem coverage, we manually develop seven applications based on open-source projects, spanning practical domains such as lifestyle sharing and travel planning. To better capture real-world user needs, we construct 300 tasks across four difficulty tiers, ranging from simple atomic actions to highly complex multi-step workflows. Together, these applications and tasks provide a more comprehensive setting for evaluating mobile agents under diverse and demanding conditions. We evaluate eight frontier models on GMA, and results show that current mobile agents remain far from reliably handling realistic user requirements, even when they perform well on atomic interactions. Beyond model evaluation, we conduct controlled ablation studies on several agent harness choices, including context retention and explicit state tracking, under a shared environment, model setting, and task taxonomy. Our results show that appropriate harness design can meaningfully improve agent performance, with particularly clear benefits on demanding workflows, while the effectiveness of specific designs can also vary across foundation models. Overall, GMA complements existing benchmarks by expanding both the breadth of the evaluated application ecosystem and the depth of task complexity. It exposes important limitations of current mobile agents and provides a challenging testbed for both evaluating model capabilities and studying how agent harness design can support reliable execution in complex mobile workflows.

1 INTRODUCTION

Recent advances in large language models (LLMs) have empowered agents to move beyond static language tasks and interact autonomously with external environments (Wei et al., 2025; Merrill et al., 2026; Huang & Jiang, 2026). Among diverse scenarios, graphical user interfaces (GUIs) (Xie et al., 2024; Yuan et al., 2026) and mobile use (Rawles et al., 2024; Kong et al., 2026) serve as an important testbed for evaluating agent capabilities. Successfully navigating GUI tasks requires agents to per-

[†] Corresponding Authors. Code Available at <https://github.com/Tongyi-Zhiwen/GMA>.

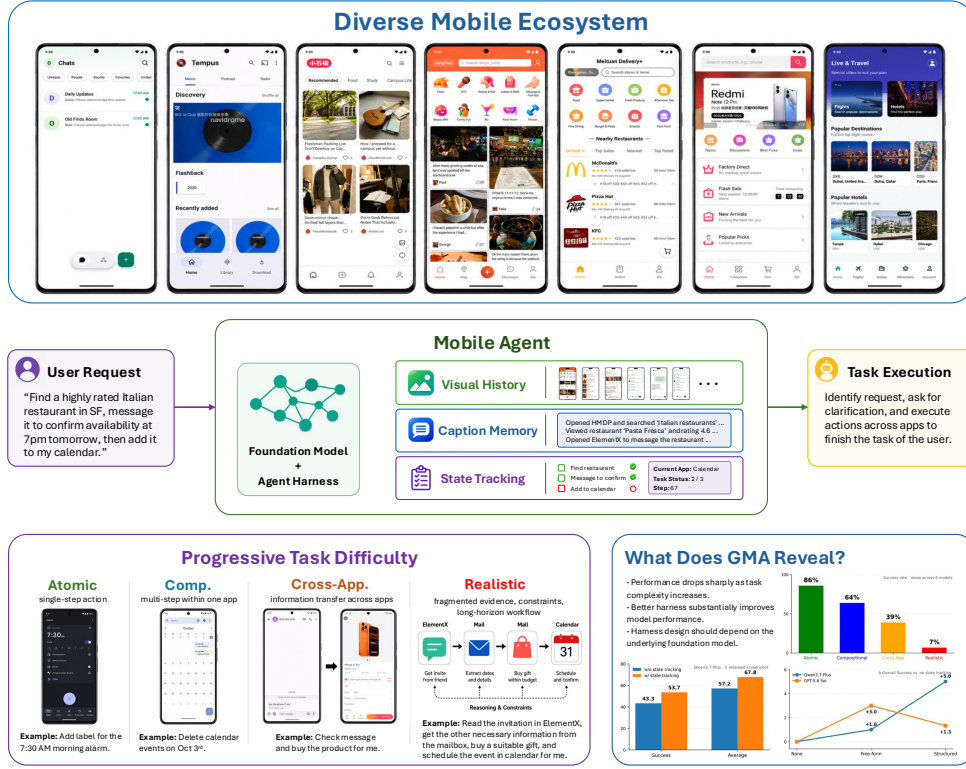


Figure 1: **Overview of GMA.** GMA provides a diverse and reproducible mobile ecosystem with seven custom applications, together with 300 tasks spanning four progressively more demanding tiers: Atomic, Compositional, Cross-Application, and Realistic. We further study the role of the agent harness, including context retention, explicit state tracking, and model-specific harness design. Evaluation on GMA reveals a substantial performance degradation as task complexity increases and highlights the growing importance of effective harness design for complex mobile workflows.

ceive visual layouts, comprehend element functions, translate high-level user intentions into precise action sequences, and adapt to dynamic application states. Furthermore, realistic mobile workflows typically demand long-horizon planning, information retrieval, constraint satisfaction, and error recovery. Such long interaction trajectories also place substantial demands on how agents preserve historical observations, intermediate information, and task progress throughout execution (Li et al., 2025; Shi et al., 2026). Together, these characteristics make GUI environments an ideal setting for assessing the capability of agents to reliably execute complex, real-world tasks.

Existing benchmarks have established a strong foundation for evaluating mobile agents within executable Android environments. AndroidWorld (Rawles et al., 2024) introduces a reproducible evaluation framework with dynamically instantiated tasks and state-based verification, serving as a canonical benchmark for mobile-agent evaluation. As a step forward, MobileWorld (Kong et al., 2026) extends AndroidWorld by integrating three additional applications and designing long-horizon, cross-application tasks. It also incorporates user interaction and MCP-augmented tasks to move beyond purely GUI-driven operations. These two benchmarks have substantially advanced the evaluation of mobile agents, and provided essential infrastructure for studying increasingly capable models.

Building on this progress, we explore how mobile agent evaluation can be further expanded along two dimensions. First, we aim to broaden application coverage so that the evaluation environment reflects a wider range of functional domains, interface designs, and interaction patterns characteristic of real Android ecosystems. Second, we seek to expand task complexity by introducing workflows that capture additional characteristics of practical mobile use, which include multi-step requirements, cross-state dependencies, and the integration of information retrieval, reasoning, and execution. These extensions provide a broader setting for uncovering the capabilities and limitations of current mobile agents.

To this end, we present GMA, a benchmark for evaluating general mobile assistants in challenging and complex mobile scenarios. To construct a functionally diverse and highly interactive Android environment, we develop seven custom applications based on open-source projects. These applications model common categories of digital services, including instant messaging, music library management, lifestyle sharing, local discovery, on-demand delivery, e-commerce, and travel planning. Together, they substantially broaden the range of mobile functions and interaction patterns available within a reproducible evaluation environment. Within this ecosystem, we meticulously design 300 tasks categorized into four difficulty tiers. This progressive structure evaluates capabilities ranging from atomic operations within a single interface to complex multi-step workflows requiring long-horizon planning, information synthesis, and cross-application reasoning. It allows us to systematically analyze capability bottlenecks as task complexity increases, while also providing a common setting for studying how different agent mechanisms behave across increasingly demanding tasks.

We evaluate several frontier model families, including Qwen, GPT and Claude series, on GMA. Our empirical analysis reveals that even models demonstrating strong proficiency on simpler tasks still struggle significantly on the more demanding workflows in our environment, and agent performance consistently degrades as task complexity increases. While the strongest models achieve high success rates on atomic interactions, no evaluated model successfully completes more than 20% of the realistic tasks, and the highest Average score on this tier is only 45.27. These results highlight the substantial difficulty of reliably executing intricate, real-world mobile workflows.

We further conduct controlled ablation studies to examine how agent harness design influences mobile task execution. Although previous GUI agents have incorporated mechanisms such as planning, reflection, and memory, their effects are often evaluated as part of complete systems in which multiple components vary simultaneously. We instead isolate several context management and state tracking choices under a shared environment and task taxonomy. Our results show that retaining richer historical information, including previous visual states and compact captions of discarded observations, can improve agent performance, with particularly clear benefits on demanding workflows. Explicitly maintaining task requirements and execution progress can also provide additional gains especially for realistic tasks. We further observe that the same state tracking design can affect different foundation models differently, suggesting that harness mechanisms may not transfer uniformly across models. Overall, these findings indicate that performance on complex mobile tasks depends not only on the capabilities of the underlying foundation model, but can also be meaningfully influenced by how the surrounding agent harness manages historical context and execution state, motivating further study of harness design alongside continued advances in foundation models.

2 RELATED WORK

2.1 GUI AND MOBILE EVALUATION

Agent evaluation has evolved from static prediction tasks toward interactive execution in dynamic environments. In the web domain, WebShop (Yao et al., 2022a) and WebArena (Zhou et al., 2023) evaluate whether agents can navigate websites and complete user-specified tasks through sequences of grounded actions. In software engineering, SWE-bench (Jimenez et al., 2024) and SWE-Bench Pro (Deng et al., 2025) assess agents on resolving repository-level issues, while ProgramBench (Yang et al., 2026) evaluates whether agents can reconstruct complete programs from their documentation and observable behavior. BrowseComp (Wei et al., 2025) and DeepResearch Bench (Du et al., 2025) instead focus on information-seeking agents, testing persistent web search, evidence collection, and the synthesis of research reports. More recently, Workspace-Bench (Tang et al., 2026) and AutomationBench (Shepard & Salimans, 2026) place agents in realistic workspace and business-process environments, requiring them to reason over heterogeneous files or coordinate operations across multiple software services. Together, these benchmarks illustrate the increasingly broad scope of agent evaluation, spanning web navigation, software engineering, information research, and workplace automation.

Among these settings, graphical user interfaces have emerged as a particularly important scenario because they require agents to jointly perceive visual content, ground language instructions to interface elements, plan action sequences, and adapt to changing application states. Early GUI evaluation primarily focused on isolated perception and action prediction capabilities in static or offline settings.

ScreenSpot (Cheng et al., 2024), for example, measures whether a model can locate the interface element corresponding to a textual instruction from a single screenshot. Android in the Wild (Rawles et al., 2023) evaluates action prediction over recorded mobile interaction trajectories. Although these benchmarks provide important diagnostics of visual grounding and interface understanding, they do not fully capture closed-loop interaction, where each action changes the subsequent environment state and agents must respond to execution errors and unexpected outcomes. More recent agentic benchmarks therefore evaluate end-to-end task completion in executable GUI environments. OSWorld (Xie et al., 2024) provides a unified environment for agents to interact with real desktop applications and operating system interfaces, using task-specific initialization and execution-based verification. AndroidWorld (Rawles et al., 2024) establishes a corresponding evaluation framework for mobile agents, featuring dynamically instantiated tasks, direct interaction with an Android emulator, and programmatic state-based verification. These two benchmarks have become canonical benchmarks for evaluating modern agents in desktop and mobile GUI environments.

Recent mobile benchmarks have extended interactive evaluation along complementary dimensions. MobileWorld (Kong et al., 2026) introduces longer cross-application workflows, agent-user interaction, and MCP-augmented tasks within a reproducible Android environment. MobileBench-OL (Wu et al., 2026b) evaluates online execution, reasoning, exploration, and robustness to environmental noise across a large collection of Chinese applications. PSPA-Bench (Nie et al., 2026) focuses on personalized mobile assistance through user-specific instructions and fine-grained process evaluation. AndroidDaily (Sui et al., 2026) evaluates agents on closed-source commercial applications using observable interaction trajectories to assess task completion without access to internal application states. MobileGym (Wu et al., 2026a) develops a lightweight, browser-hosted mobile simulation platform with several web applications that simulate real mobile apps, supporting deterministic state-based evaluation and highly parallel rollouts for agent training. iOSWorld (Jang et al., 2026) extends interactive mobile evaluation beyond Android by introducing a native iOS environment with persistent user data and tasks requiring personalization and cross-application reasoning. Together, these benchmarks broaden mobile-agent evaluation across application coverage, personalization, platform diversity, environmental realism, and training scalability.

2.2 AGENT HARNESS

The concept of agent harness has recently been used to describe the system surrounding a foundation model that enables and structures its interaction with tools and environments (Anthropic, 2025a;b). The underlying idea, actually, is not new in agent research, as many earlier systems had already incorporated mechanisms that organize the interaction loop, manage context, and connect models with external actions. ReAct (Yao et al., 2022b) interleaves reasoning, actions, and environment observations, providing a general framework task solving agents. Reflexion (Shinn et al., 2023) further augments agents with linguistic feedback, self-reflection, and episodic memory to improve behavior across repeated attempts. In software engineering, SWE-agent (Yang et al., 2024) introduces an agent-computer interface for repository navigation, file editing, and test execution. OpenHands (Wang et al., 2025) provides a broader platform integrating code editing, terminal execution, web browsing, and sandbox management. Viewed through the contemporary notion of the agent harness, these works demonstrate how external scaffolding can organize model behavior and adapt foundation models to complex interactive tasks.

Following the introduction of this terminology, recent work has increasingly treated the harness as a first-class component that can be deliberately designed, evaluated, and improved. OpenAI (2026) demonstrates how repository structure, documentation, testing infrastructure, observability, and feedback loops can make a large software project legible to coding agents and progressively increase their autonomy. Lin et al. (2026) introduces an observability-driven framework that automatically evolves harness components such as tools, middleware, and long-term memory from execution experience. Harness-Bench (Team, 2026) systematically compares configuration-level harness effects under shared tasks and execution conditions, showing that agent performance should not be attributed to the foundation model alone.

For GUI agents, harness plays an important role in coordinating perception, planning, memory, verification, and execution over long interaction trajectories. HiconAgent (Zhou et al., 2026) improves the use of interaction history through dynamic context sampling and history compression, reducing computational overhead while preserving task-relevant information. Mobile-Agent-v3.5 (Xu et al.,

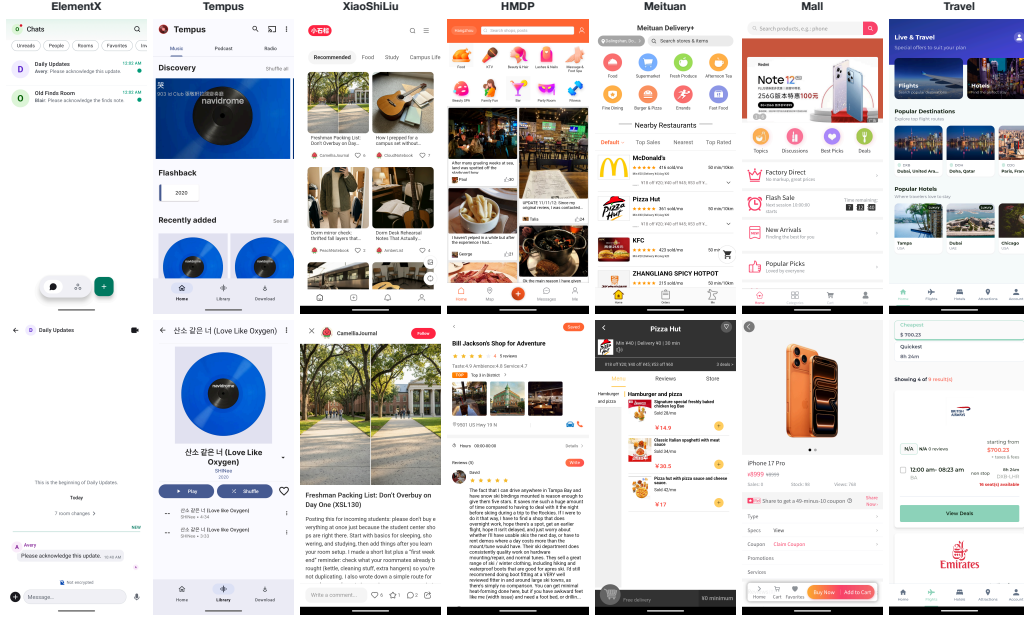


Figure 2: Screenshots of the seven custom applications developed for GMA. From left to right, the applications cover instant messaging, music library, lifestyle sharing, local discovery, food delivery, e-commerce, and travel planning, providing diverse interfaces, interaction patterns, and application-specific workflows for mobile agent evaluation.

2026) employs a multi-agent harness that assigns planning, execution, action verification, and persistent memory to specialized Manager, Worker, Reflector, and Notetaker roles. PhoneHarness (Li et al., 2026) coordinates GUI interactions, device-side commands, and structured tools through a unified execution loop with deterministic action routing and auditable traces. Zheng et al. (2026) introduces a training-free external wrapper that maintains a global instruction summary, a dynamic subgoal-progress tracker, and a transition-aware action verifier to guide the reasoning process of GUI agents. Together, these works demonstrate the importance of structured planning, historical context, explicit memory, and task-state management in GUI agent harnesses. While existing studies demonstrate the benefits of particular harness mechanisms, their effects are often studied within different agent systems and evaluation settings. This leaves room for more controlled analysis of how specific harness choices affect performance under a shared environment and how these effects vary with task complexity and the underlying foundation model.

3 GMA ENVIRONMENT

3.1 APPLICATIONS

To evaluate mobile agents across a broad range of interactions, the underlying environment should cover diverse functional domains, interface structures, and application-specific workflows. Consequently, we expand the benchmark scope by introducing seven custom applications, where each represents a common category of mobile service, deliberately chosen to introduce unique interaction paradigms and challenge distinct facets of the agent reasoning and execution capabilities:

- **ElementX¹** serves as a comprehensive instant messaging client built upon the Matrix ecosystem. It supports direct and group conversations, public and private rooms, message editing, file sharing, and interactive polls. This feature density facilitates tasks requiring interpersonal communication, information aggregation, and collaborative coordination.

¹Android client source: <https://github.com/element-hq/element-x-android>. Server source: <https://github.com/element-hq/dendrite>.

- **Tempus**² is a music library application linked to a self-hosted Navidrome server. Agents can navigate extensive catalogs of songs, albums, and artists, as well as curate playlists and manage favorites. These mechanics provide an ideal testbed for tasks centered on media retrieval, structured comparison, and the logical organization of digital collections.
- **XiaoShiLiu**³ models a multimodal lifestyle sharing platform driven by user-generated content. It features categorized images, nested comment threads, likes, content collections, and social following systems. This environment requires agents to parse unstructured social content, extract personalized recommendations, and execute multi-stage social interactions.
- **HMDP**⁴ acts as a crowdsourced local discovery and review hub. Agents can search for businesses, scrutinize user reviews, and publish their own experience posts. Additional features like shop favoriting, user following, and voucher management challenge agents to synthesize subjective opinions and navigate localized service ecosystems.
- **Meituan**⁵ replicates an on-demand food delivery service. It encompasses restaurant and menu browsing, address management, cart configurations, ordering workflows, and post-order feedback. Navigating this application requires the agent to handle transactional sequences, spatial constraints, and multi-step checkout procedures.
- **Mall**⁶ provides a general e-commerce environment. The application supports granular product searches, category browsing, variant selection, shopping cart management, checkout processes, and order tracking. These features rigorously test the capability to navigate hierarchical catalogs, manage user constraints, and confirm commercial transactions.
- **Travel**⁷ delivers a multi-domain travel planning service that encompasses independent domains for flights, hotels, and tourist attractions. It supports complex search and comparison logic, passenger data entry, secure booking, and payment flows. Navigating these distinct booking pipelines challenges agents with constraint-heavy planning, scheduling dependencies, dense information retrieval, and the meticulous completion of extensive, multi-field forms.

While these applications provide the necessary functional breadth, integrating them into a rigorous benchmark requires strict determinism. Rather than relying on simplified mock interfaces, we adapt these applications directly from fully functional open-source projects to preserve authentic interaction flows. To eliminate the variability and unreliability of live external services, we completely decouple all applications from production APIs and we self-host all necessary backend services and databases, populating them with rich, locally managed datasets. Furthermore, we engineer programmatic interfaces to precisely initialize, track, and verify application states during testing, ensuring a fully reproducible evaluation setting.

Beyond these seven custom additions, GMA incorporates standard Android system utilities to provide essential foundational functionality. We also retain Mattermost and Mastodon from MobileWorld (Kong et al., 2026), supplying enterprise collaboration and decentralized microblogging features. Together, these applications constitute a cohesive and highly diverse mobile ecosystem, and this expansive breadth enables us to design sophisticated, cross-application workflows. Rather than evaluating agents only within isolated applications, GMA includes workflows requiring information transfer, state tracking, and sequential execution across functionally distinct domains, thereby capturing important compositional characteristics of practical mobile use.

3.2 INFRASTRUCTURE

Inherited Foundation. We build the foundation of GMA upon the rigorous evaluation principles and containerized architectures already established by AndroidWorld (Rawles et al., 2024) and

²Android client source: <https://github.com/eddyizm/tempus>. Music server source: <https://github.com/navidrome/navidrome>.

³Project source: <https://github.com/ZTMYO/XiaoShiLiu>.

⁴Project source: <https://github.com/java-up-up/hmdp-plus>.

⁵Web client source: <https://github.com/zwStar/vue-meituan>. Server source: <https://github.com/zwStar/meituan-backend>.

⁶Web client source: <https://github.com/macrozheng/mall-app-web>. Server source: <https://github.com/macrozheng/mall>.

⁷Project source: <https://github.com/mojahidhasan/fullstack-nextjs-golobe-travel-agency>.

MobileWorld (Kong et al., 2026). From these frameworks, we inherit fully containerized emulator infrastructures that utilize system snapshots to guarantee reproducible starting environments, alongside objective, state-based verification mechanisms that assess task success without relying on brittle visual heuristics. Extending these robust baselines, our infrastructure development focuses on scaling this environment to accommodate a significantly broader suite of self-hosted applications and the more demanding tasks.

Unified State Abstraction. To seamlessly coordinate interactions across these varied functional domains, we introduce a unified, declarative asset abstraction framework. This approach allows heterogeneous application data to become fully composable. An asset represents a typed, modular piece of task state, such as a user contact, message thread, calendar event, social post, e-commerce order, or travel booking. A task specifies its initial conditions as a collection of these assets, which the environment dynamically materializes post-reset via application-specific adapters. This identical representation is then reused to define expected final states, providing a standardized interface for constructing and evaluating tasks across both native on-device applications and our self-hosted services. By decoupling high-level task semantics from low-level database schemas, this design drastically reduces the engineering overhead required to construct our sophisticated scenarios.

Application State Adapters. To operationalize this asset framework within our custom applications, we develop dedicated state adapters for each of the seven new functional domains. Every adapter exposes standardized programmatic operations for creating, querying, and resetting evaluation-relevant entities. Crucially, because our self-hosted backends and the Android client interfaces maintain state independently, simply restoring a frontend emulator snapshot is insufficient for full reproducibility. To resolve this discrepancy, our task initialization pipeline synchronizes snapshot restoration with targeted backend resets, cache clearance, session invalidation, and deterministic asset injection.

Composable State Verification. Building upon this precise state-alignment infrastructure, we implement a highly reusable verification framework that programmatically tests whether specific assets exist, have been modified, or have been successfully deleted. By defining these state changes as individual criteria, a single task can flexibly combine and weight them, yielding both a strict binary success signal and a fine-grained continuous score for partially completed workflows. Ultimately, this architecture ensures that the exact same objective evaluation interface scales from atomic, single-application actions to long-horizon workflows spanning multiple domains.

Deterministic Text Verification. For tasks requiring text entry or explicit answer submission, we intentionally avoid non-deterministic evaluation mechanisms such as semantic matching or LLM-as-a-Judge. Instead, we explicitly specify the expected formatting constraints directly within the task prompt and enforce strict, deterministic verification logic. This design provides stable and reproducible evaluation across models and test runs while avoiding variability introduced by model-based judges. It also imposes a higher operational bar on the agent, testing its ability to adhere strictly to specified formatting contracts and output precise deliverables rather than relying on loose, approximate semantic equivalence.

3.3 TASK DESIGN

To systematically expose the capability boundaries of current models, we stratify our benchmark into four progressive difficulty tiers based on operation count, cross-application dependencies, information synthesis requirements, and context ambiguity.

- **Atomic tasks** isolate single-step operations with explicitly defined target states, establishing a baseline for GUI element perception and fundamental control execution.
- **Compositional tasks** require chaining multiple operations typically within a single application, evaluating short-horizon planning, basic information retrieval, and state tracking.
- **Cross-application tasks** introduce strict inter-application dependencies, where information synthesized in one environment dictates actions in another, demanding long-horizon planning, working memory, and precise cross-domain information transfer.

- **Realistic tasks** represent the most challenging tier, inheriting cross-application dependencies while further introducing underspecified requirements and fragmented evidence across four to seven applications. Agents must parse fragmented evidence amidst distractors, reconcile competing constraints, and execute causally dependent actions. For instance, a typical realistic task requires deducing a product category from an ElementX poll, extracting budget limits from Mail, comparing eligible items in Mall, and scheduling delivery in Calendar.

To evaluate this compositionality with rigorous granularity, we structurally decompose each task into one or more discrete subtasks. A subtask represents a specific, independently verifiable milestone within the broader user requirement, such as successfully drafting an email, finalizing a cart checkout, or extracting a precise piece of data. By breaking down complex instructions into a verifiable checklist of these constituent subtasks, the benchmark can explicitly measure partial progress through long-horizon scenarios, directly supporting the continuous scoring metrics defined in our evaluation protocol rather than relying solely on monolithic binary outcomes.

While the difficulty tiers and subtask formulations capture the structural complexity of a workflow, we additionally consider five non-exclusive dimensions during task design. These dimensions provide lightweight guidance for constructing tasks with diverse interaction and reasoning requirements, and also help characterize some of the challenges involved. They are not used as mandatory per-task tags. A task may involve multiple dimensions, while many tasks are not explicitly associated with any of them:

- **Multi-Step Workflow** captures tasks that require completing a single overarching objective through a coordinated sequence of actions across multiple apps, where information or outcomes from earlier steps are needed in subsequent steps.
- **Selection & Optimization** captures scenarios where agents must compare multiple feasible alternatives and select the option that best satisfies a set of constraints or preferences.
- **Information Gathering** captures tasks that require actively locating, interpreting, and synthesizing information from the environment rather than receiving all necessary parameters directly from the original user instruction.
- **Conditional** captures workflows whose correct execution depends on observed conditions, requiring the agent to select, modify, or terminate a workflow according to the applicable branch.
- **Invalid Instruction** captures requests that cannot be completed as stated because they are infeasible, unsupported, or missing essential information, which requires the agent to recognize the limitation and respond appropriately by requesting clarification or proposing a valid alternative.

Guided by these criteria, the finalized benchmark comprises a total of 300 meticulously curated tasks, encompassing 73 Atomic, 129 Compositional, 78 Cross-application, and 20 Realistic tasks. To guarantee rigor and reproducibility, every task undergoes a strict dual-annotator quality control process. This review systematically verifies the semantic clarity of the instruction, the accuracy of its assigned difficulty and dimension metadata, and the logical alignment between the intended user requirement and the programmatic verification criteria for each underlying subtask.

4 EVALUATION ON GMA

4.1 EXPERIMENTAL SETTINGS

Action Space. We anchor our action space in the established MobileWorld framework (Kong et al., 2026), adopting its standard GUI operations and navigation commands. To accommodate the demanding evaluation dimensions of GMA, we also include `call_user` and `answer`. The `call_user` action enables the agent to query a simulated user when critical instruction parameters are missing or ambiguous. Conversely, the `answer` action allows the agent to explicitly submit text-based deliverables when required by the task instruction. We also include a standard `terminate` action for the agent to declare that it has finished its execution.

Agent Architecture and Context Management. For our baseline evaluations, we adopt a straightforward, zero-shot autoregressive agent design. Notably, we enforce a strict vision-only observation paradigm where the agent receives only raw screenshots representing the visual state of the device,

Table 1: Overall and difficulty-specific performance of evaluated models on GMA. For each setting, we report the strict task success rate (*Success*) and the average proportion of completed subtasks (*Average*). The best result is highlighted in bold, and the second-best result is underlined.

Models	Overall		Atomic		Compositional		Cross-Application		Realistic	
	<i>Success</i>	<i>Average</i>	<i>Success</i>	<i>Average</i>	<i>Success</i>	<i>Average</i>	<i>Success</i>	<i>Average</i>	<i>Success</i>	<i>Average</i>
Doubao-Seed-2.1-pro	62.00	74.42	<u>89.04</u>	<u>89.73</u>	65.89	79.27	41.03	62.99	20.00	31.75
Qwen3.7-Plus	49.33	63.78	78.08	80.37	51.94	69.78	30.77	51.87	0.00	11.00
Qwen3.8-Max	67.33	80.35	93.15	93.84	70.54	85.17	52.56	70.57	10.00	38.22
Gemini 3.5 Flash	54.33	65.91	80.82	81.74	62.02	75.23	30.77	50.64	0.00	7.56
GPT-5.5	59.67	71.14	87.67	87.67	65.12	78.71	39.74	58.78	0.00	10.25
GPT-5.6 Sol	<u>62.67</u>	<u>75.86</u>	86.30	86.99	<u>68.99</u>	<u>81.60</u>	<u>43.59</u>	<u>64.21</u>	10.00	<u>43.64</u>
Claude Sonnet 4.6	55.33	68.13	83.56	83.56	60.47	75.27	34.62	50.33	0.00	35.06
Claude Opus 4.7	60.00	72.33	87.67	87.67	65.12	78.01	37.18	55.54	<u>15.00</u>	45.27

with zero access to the underlying XML accessibility trees or structural UI hierarchies. Throughout a task, the agent maintains a continuous textual interaction history, accumulating its complete trajectory of reasoning traces and executed actions. To mitigate visual context bloat, we implement a sliding window for visual observations, retaining only the two most recent screenshots while purging older images from the prompt. When the agent utilizes the `call_user` action, the response is injected into the subsequent observation state as authoritative context and is permanently retained within the textual history.

User Simulation. To evaluate interactive workflows without human-in-the-loop bottlenecks, we deploy Qwen3.7-Plus (Qwen Team, 2026a) as an automated user simulator. To prevent the simulator from inadvertently trivializing the task, it operates under a strict task-specific interaction contract. This prompt-driven contract restricts the simulator to only the information explicitly assigned to the user persona for that scenario, strictly prohibiting it from providing unsolicited operational guidance or revealing step-by-step solutions.

Evaluation Protocol and Metrics. We impose a strict computational budget of 150 interaction steps per episode. An evaluation run terminates under one of three conditions: the agent emits an explicit `terminate` or `answer` action, the task verification engine confirms early success, or the maximum step limit is reached. Upon termination, we evaluate the model performance using two primary metrics: *Success* rate and *Average* score. Because tasks frequently comprise multiple constituent subtasks, the strict *Success* rate is a binary metric awarded only when all subtasks within a given task are perfectly satisfied. To capture granular progress on long-horizon workflows, the *Average* score measures the proportion of correctly completed subtasks within a task.

4.2 RESULTS AND ANALYSES

We evaluate eight frontier models from five model families: Doubao-Seed-2.1-pro (ByteDance Seed, 2026) from ByteDance Seed; Qwen3.7-Plus (Qwen Team, 2026a) and Qwen3.8-Max (Qwen Team, 2026b) from Alibaba Qwen; Gemini 3.5 Flash (Google DeepMind, 2026) from Google DeepMind; GPT-5.5 (OpenAI, 2026a) and GPT-5.6 Sol (OpenAI, 2026b) from OpenAI; Claude Sonnet 4.6 (Anthropic, 2026b) and Claude Opus 4.7 (Anthropic, 2026a) from Anthropic. Table 1 reports both *Success* rate and *Average* score for each model across the four difficulty tiers.

Overall performance. Qwen3.8-Max achieves the strongest overall performance, reaching a *Success* rate of 67.33 and an *Average* score of 80.35. It also ranks first on both metrics across the atomic, compositional, and cross-application tiers, demonstrating consistently strong performance on tasks up to substantial compositional complexity. GPT-5.6 Sol emerges as the strongest runner-up, achieving the second-best overall *Success* rate of 62.67 and *Average* score of 75.86. It also

maintains competitive performance across different difficulty tiers, consistently ranking among the top-performing models as task complexity increases. Doubao-Seed-2.1-pro follows closely, obtaining an overall *Success* rate of 62.00 and an *Average* score of 74.42.

Performance across difficulty levels. All models exhibit a clear performance decline as task difficulty increases. On atomic tasks, the strongest model achieves 93.15 and 93.84 in *Success* and *Average* respectively, and all models obtain *Average* scores above 80. Performance decreases considerably on cross-application tasks, where Qwen3.8-Max reaches only 52.56 in *Success*. This consistent degradation indicates that composing more operations, preserving intermediate information, and managing complex dependencies remain challenging even for the strongest models.

Challenges in realistic workflows. The decline becomes particularly pronounced on the realistic tier. No model completes more than 20% of these tasks perfectly, with Doubao-Seed-2.1-pro achieving the highest *Success* rate of 20.00. Claude Opus 4.7 achieves the highest *Average* score of 45.27, followed by GPT-5.6 Sol with 43.64, while the overall leader Qwen3.8-Max obtains 38.22. The substantial gap shows that models can often complete individual parts of these workflows but struggle to satisfy all interdependent requirements within a single episode.

Different strengths on complex tasks. The model ranking on realistic tasks differs notably from the overall performance. Doubao-Seed-2.1-pro achieves the highest *Success* rate, while Claude Opus 4.7 achieves the highest *Average* score, despite neither model leading the overall results. Conversely, Qwen3.8-Max dominates the first three difficulty tiers but does not retain the same advantage on realistic tasks. These results suggest that strong performance on atomic and moderately compositional interactions does not necessarily translate to reliable execution of holistic mobile workflows involving long-horizon reasoning, information synthesis, and multiple cross-application dependencies.

4.3 PERFORMANCE ACROSS APPLICATIONS

We further examine how agent performance varies across the application ecosystem, and Figure 3 reveals substantial variation. Travel is particularly challenging, with an *Average* score of only 20.17, followed by XiaoShiLiu at 55.00. Several other applications introduced in GMA, including Mail and ElementX, also fall toward the lower end of the performance range. Meanwhile, some newly introduced applications, such as Meituan and HMDP, achieve substantially stronger results. These observations suggest that the difficulty introduced by broader application coverage is not uniform, but several of the weakest-performing scenarios do arise from applications newly covered by our benchmark.

This variation highlights an important challenge in evaluating mobile agents across a broader application ecosystem. Strong performance on commonly evaluated applications does not necessarily translate uniformly to all additional scenarios, and current agents can still experience substantial degradation when interacting with different application interfaces, workflows, and task settings. The low performance observed on several newly introduced applications therefore provides evidence that current capabilities have not yet generalized reliably across the full range of mobile environments considered in GMA. Expanding application coverage is thus important not only for more comprehensive evaluation, but also for developing and training mobile agents that can generalize more reliably across diverse interfaces and usage scenarios.



Figure 3: Performance across different applications, averaged over all evaluated models. Light and dark bars denote *Success* rate and *Average* scores, respectively. Orange denotes applications introduced in GMA, blue denotes applications inherited from MobileWorld, and green denotes Android system applications.

5 HARNESS MATTERS FOR GMA TASKS

Harness design has emerged as an important topic in agent research, with growing evidence that an effective harness can substantially unlock model capabilities in domains such as software engineering. However, comparatively little work has examined how harness design could influence performance for mobile agents, especially when tasks require sustained interaction over long and complex trajectories. This motivates us to take a closer look at the role of harness design in GMA, with the goal of understanding not only whether different mechanisms help, but also how their effects vary and interact under increasingly demanding mobile tasks.

5.1 CONTEXT RETENTION

We first study how the amount of retained visual context affects agent performance. As shown in Table 2, increasing the number of historical screenshots generally improves performance, especially when moving from a very limited visual context to a moderate one. With only one screenshot retained, Qwen3.7-Plus achieves an overall *Success* rate of 43.33 and *Average* score of 57.20, while retaining eight screenshots increases these scores to 52.00 and 67.10 respectively. The improvement is also pronounced on realistic tasks, where the average score rises from 9.97 to 21.44. However, the marginal benefit gradually diminishes as more screenshots are added, and retaining 16 images does not bring further overall improvement, with the *Average* score slightly decreasing. This suggests that historical visual observations are useful for preserving task-relevant information, but simply enlarging the visual context is insufficient once a moderate amount of history is already available.

Retaining previous reasoning traces provides another important source of historical information. Under the same two-image setting, removing previous thoughts decreases the *Success* rate from 49.33 to 41.67 and the *Average* score from 63.78 to 55.77. With only a small number of screenshots retained, removing the reasoning history effectively discards most information about earlier interaction steps, leaving the agent to rely largely on the current interface state and a very limited visual history. The impact becomes more evident on more complex tasks. *Average* performance drops from 69.78 to 57.13 on compositional tasks, from 51.87 to 43.03 on cross-application tasks, and from 11.00 to 6.89 on realistic tasks. We further examine an alternative strategy that replaces discarded historical screenshots with textual captions. Under the two-image setting, retaining captions for older screenshots improves the overall *Success* rate from 49.33 to 51.33 and the *Average* score from 63.78 to 65.13. The improvement is particularly substantial on realistic tasks, where the *Average* score increases from 11.00 to 34.60. This suggests that compact textual summaries of older visual observations can preserve useful historical information without requiring all screenshots to remain in the context. Together, these results suggest that maintaining appropriate historical context is particularly important for tasks that require agents to preserve intermediate information and track progress over long interaction trajectories.

Table 2: Effect of context retention on Qwen3.7-Plus. **Images** denotes the number of most recent screenshots retained in the context, **Thought** indicates whether previous reasoning traces are preserved, and **Caption** indicates whether older screenshots are retained as captions.

Images	Thought	Caption	Overall		Difficulty			
			Success	Average	Atomic	Comp.	Cross-App.	Realistic
1	✓	✗	43.33	57.20	72.83	61.67	47.30	9.97
2	✓	✗	49.33	63.78	80.37	69.78	51.87	11.00
2	✗	✗	41.67	55.77	80.37	57.13	43.03	6.89
2	✓	✓	51.33	65.13	78.31	70.65	51.50	34.60
4	✓	✗	51.00	65.53	80.37	71.89	54.64	12.86
8	✓	✗	52.00	67.10	83.56	72.32	54.77	21.44
16	✓	✗	52.00	65.52	79.68	70.95	53.97	23.81

5.2 EXPLICIT STATE TRACKING

Modern agents commonly maintain explicit memory or task status to preserve critical information across long interaction trajectories. For mobile agents, this becomes particularly important as relevant information may disappear from the current screen while task progress accumulates across multiple actions and applications. To study the effect of such structured memory, we implement an explicit state tracking mechanism with reference to the Task-State Representation (TSR) framework proposed in Zheng et al. (2026), maintaining task requirements, subgoal progress, and execution state throughout the trajectory. We compare agents with and without this mechanism under different amounts of retained visual history.

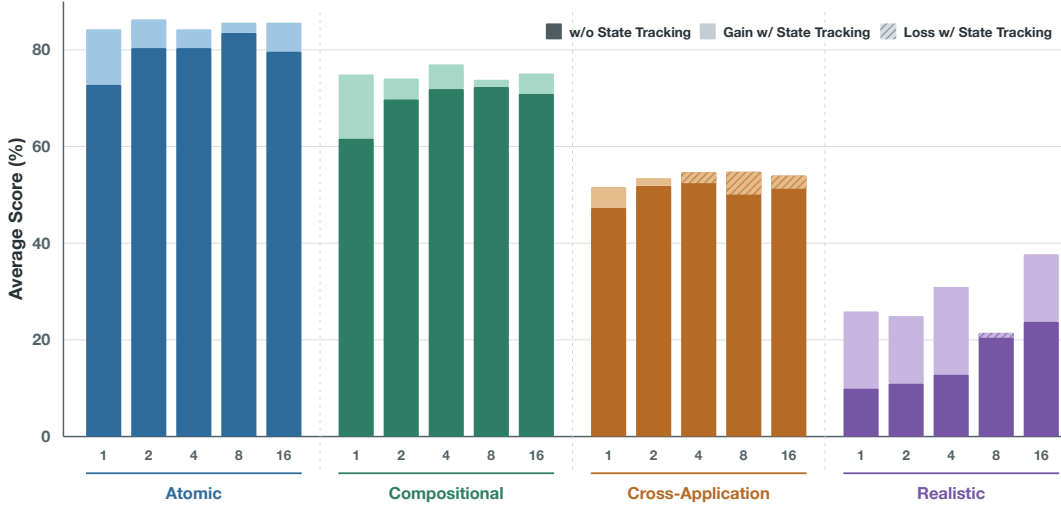


Figure 5: Effect of state tracking across different difficulty tiers. The solid portion denotes the *Average* score without state tracking, while the lighter or hatched portion indicates the gain or loss after enabling state tracking. Numbers below each group denote the number of retained screenshots.

Figure 4 shows that explicit state tracking generally improves agent performance across different visual context sizes. The improvement is particularly clear when visual history is limited. With one retained screenshot, state tracking increases *Success* rate from 43.33 to 53.67 and the *Average* score from 57.20 to 67.85. More broadly, it improves the *Success* rate under all tested context sizes and improves the *Average* score in most settings. The benefit becomes smaller as more screenshots are retained, and with eight screenshots the *Average* score remains nearly unchanged, decreasing slightly from 67.10 to 66.95. These results indicate that explicitly maintaining task state can provide useful information beyond simply retaining raw interaction history, particularly when the available visual history is limited.

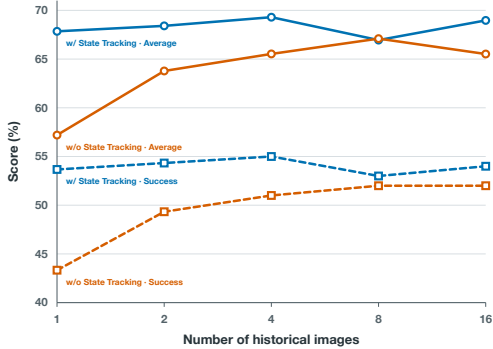


Figure 4: Overall performance with and without explicit state tracking under different numbers of retained screenshots.

Results in Figure 5 show that the effect of state tracking differs substantially across difficulty tiers. On atomic and compositional tasks, state tracking consistently improves the *Average* score across all tested visual context sizes. For example, with one retained screenshot, the score increases from 72.83 to 84.25 on atomic tasks and from 61.67 to 74.90 on compositional tasks. Positive gains remain even when more screenshots are retained, indicating that explicit task state provides useful information beyond raw visual history for these tasks. The effect is less consistent on cross-application tasks. State tracking improves performance when only one or two screenshots are retained, but slightly decreases the *Average* score with four, eight, and sixteen screenshots. In contrast, the strongest benefits appear on realistic tasks. State tracking increases the *Average* score from 9.97 to 25.89 with one screenshot, from 12.86 to 30.97 with four screenshots, and from 23.81 to 37.72 even with sixteen screenshots. The eight-image setting is the only exception, where performance decreases slightly from 21.44 to 20.42. Overall, these results suggest that explicitly organizing task requirements and execution progress is particularly valuable for realistic workflows.

The amount of retained visual context further modulates the benefit of state tracking. For the first three difficulty tiers, the improvement brought by state tracking generally becomes smaller as more historical screenshots are retained, suggesting that a richer visual context can already preserve part of the information that explicit state tracking provides. This trend is particularly evident compared

with the large gains observed under the one-image setting. Realistic tasks, however, exhibit a notably different pattern. Even when sixteen screenshots are retained, state tracking still increases the *Average* score from 23.81 to 37.72, corresponding to a substantial gain of 13.91 points, which is only slightly smaller than the 15.92-point improvement observed with a single retained screenshot. This suggests that, for highly complex workflows, explicit state tracking can still provide valuable complementary information even when rich visual history is already available, by organizing task requirements, intermediate progress, and cross-step dependencies in a more accessible form.

5.3 MODEL-SPECIFIC HARNESS DESIGN

The effectiveness of a harness may also depend on the underlying foundation model. To examine this assumption, we compare two state-tracking designs on Qwen3.7-Plus and GPT-5.6 Sol. The *Free-form* design allows the model to maintain task state in an unconstrained textual format, whereas the *Structured* design used in our previous experiments requires the model to update a predefined set of state fields. We additionally report the performance without explicit state tracking as a reference.

The two models respond quite differently to the same state-tracking designs. For Qwen3.7-Plus, the structured representation provides a clear advantage, increasing the *Success* rate from 49.33 to 54.33 and the *Average* score from 63.78 to 68.41. The improvement is particularly large on realistic tasks, where the score rises from 11.00 without state tracking to 24.92. In comparison, the free-form representation brings only limited overall improvement and even performs worse on realistic tasks. This suggests that Qwen3.7-Plus benefits from an explicit structure that guides how task-relevant information and progress are maintained.

The same pattern does not hold for GPT-5.6 Sol. The free-form representation achieves the highest overall *Success* rate of 65.67, but its *Average* score of 75.75 remains nearly unchanged from 75.86 without state tracking. The structured representation achieves a *Success* rate of 64.00 while reducing the overall *Average* score to 73.96. Its effect also varies across difficulty tiers. The structured design slightly improves performance on atomic and compositional tasks, but decreases the *Average* score on cross-application tasks from 64.21 to 54.04 and on realistic tasks from 43.64 to 42.56. Thus, unlike Qwen3.7-Plus, GPT-5.6 Sol does not consistently benefit from adding explicit structure to its state representation.

Table 3: Effect of different state-tracking designs across models. *Free-form* allows unconstrained state representation, while *Structured* uses predefined state fields.

Model	State	Overall		Difficulty			
		<i>Success</i>	<i>Average</i>	<i>Atomic</i>	<i>Comp.</i>	<i>Cross-App.</i>	<i>Realistic</i>
Qwen	None	49.33	63.78	80.37	69.78	51.87	11.00
	Free-form	50.33	64.73	85.16	71.27	49.37	7.89
	Structured	54.33	68.41	86.30	74.07	53.46	24.92
GPT	None	62.67	75.86	86.99	81.60	64.21	43.64
	Free-form	65.67	75.75	87.67	84.11	60.32	38.54
	Structured	64.00	73.96	88.36	82.72	54.04	42.56

These results together suggest that harness mechanisms are not universally transferable across foundation models. Additional structure can effectively guide one model while unnecessarily constraining another. Harness design should therefore consider the characteristics of the underlying model rather than assuming a single configuration is optimal for all agents. We view this analysis as an initial investigation, and broader experiments across more foundation models, harness designs, and task settings are still needed to better characterize these model-harness interactions and examine how consistently they generalize across mobile scenarios.

6 CONCLUSION

We present GMA, a benchmark for evaluating general mobile assistants in challenging mobile scenarios, featuring seven custom applications, 300 tasks across four progressive difficulty tiers, and a reproducible evaluation infrastructure. Evaluation of eight frontier models shows a clear performance decline as task complexity increases, with cross-application and realistic workflows remaining particularly challenging. We further study several agent harness choices and find that context retention and explicit state tracking can meaningfully affect performance, while the same design may not benefit different foundation models equally. Overall, GMA provides a broader and more challenging testbed for evaluating mobile agents and studying how model capabilities and harness design shape performance in complex mobile environments.

REFERENCES

- Anthropic. Building agents with the claude agent sdk. <https://claude.com/blog/building-agents-with-the-claude-agent-sdk>, September 2025a.
- Anthropic. Effective harnesses for long-running agents. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>, November 2025b.
- Anthropic. System Card: Claude Opus 4.7. <https://www-cdn.anthropic.com/037f06850df7f8e871e206dad004c3db5fd50340/Claude%20Opus%204.7%20System%20Card.pdf>, April 2026a.
- Anthropic. System Card: Claude Sonnet 4.6. <https://www-cdn.anthropic.com/bbd8ef16d70b7a1665f14f306ee88b53f686aa75/Claude%20Sonnet%204.6%20System%20Card.pdf>, February 2026b.
- ByteDance Seed. Seed2.1 Officially Released: Advancing AI Productivity. <https://seed.bytedance.com/en/blog/seed2-1-officially-released-advancing-ai-productivity>, June 2026.
- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9313–9332, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.acl-long.505>.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL <https://arxiv.org/abs/2509.16941>.
- Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. Deepresearch bench: A comprehensive benchmark for deep research agents. *arXiv preprint*, 2025.
- Google DeepMind. Gemini 3.5 Flash Model Card. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-5-Flash-Model-Card.pdf>, May 2026.
- Wenqi Huang and Peter Jiang. Deepswe v1.1: a cleaner, more reproducible benchmark for frontier coding agents, 2026. URL <https://github.com/datacurve-ai/deep-swe>.
- Lawrence Keunho Jang, Mareks Woodside, Geronimo Carom, Andrew Keunwoo Jang, Jing Yu Koh, and Ruslan Salakhutdinov. iosworld: A benchmark for personally intelligent phone agents, 2026. URL <https://arxiv.org/abs/2606.09764>.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, Zhidan Liu, Steven Hoi, and Yue Wang. Mobile-world: Benchmarking autonomous mobile agents in agent-user interactive, and mcp-augmented environments. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026. URL <https://arxiv.org/abs/2512.19432>.
- Chenxin Li, Zhengyao Fang, Zhengyang Tang, Pengyuan Lyu, Xingran Zhou, Xin Lai, Fei Tang, Liang Wu, Yiduo Guo, Weinong Wang, Junyi Li, Yi Zhang, Yang Ding, Huawei Shen, Sunqi Fan, Shangpin Peng, Zheng Ruan, Anran Zhang, Benyou Wang, Chengquan Zhang, and Han Hu. Phoneharness: Harnessing phone-use agents through mixed gui, cli, and tool actions, 2026. URL <https://arxiv.org/abs/2606.14832>.

-
- Ning Li, Xiangmou Qu, Jiamu Zhou, Jun Wang, Muning Wen, Kounianhua Du, Xingyu Lou, Qiuying Peng, Jun Wang, and Weinan Zhang. Mobileuse: A hierarchical reflection-driven GUI agent for autonomous mobile operation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=KR6tnkb6h4>.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis, Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muenighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- Hongyi Nie, Xunyuan Liu, Yudong Bai, Yaqing Wang, Yang Liu, Quanming Yao, and Zhen Wang. Pspa-bench: A personalized benchmark for smartphone gui agent, 2026. URL <https://arxiv.org/abs/2603.29318>.
- OpenAI. GPT-5.5 System Card. <https://deploymentsafety.openai.com/gpt-5-5/gpt-5-5.pdf>, April 2026a.
- OpenAI. GPT-5.6 System Card. <https://deploymentsafety.openai.com/gpt-5-6/gpt-5-6.pdf>, July 2026b.
- OpenAI. Harness engineering: Leveraging Codex in an agent-first world. <https://openai.com/index/harness-engineering/>, February 2026.
- Qwen Team. Qwen3.7-Plus: Multimodal agent intelligence, May 2026a. URL <https://qwen.ai/blog?id=qwen3.7-plus>.
- Qwen Team. Qwen3.8: A new bar for coding and cowork, August 2026b. URL <https://qwen.ai/blog?id=qwen3.8>.
- Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control, 2023. URL <https://arxiv.org/abs/2307.10088>.
- Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2024. URL <https://arxiv.org/abs/2405.14573>.
- Daniel Shepard and Robin Salimans. Automationbench, 2026. URL <https://arxiv.org/abs/2604.18934>.
- Yibo Shi, Jungang Li, Linghao Zhang, Zihao Dongfang, Biao Wu, Sicheng Tao, Yibo Yan, Chenxi Qin, Weitong Liu, Zhixin Lin, Hanqian Li, Yu Huang, Song Dai, Yonghua Hei, Yue Ding, Xiang Li, Shikang Wang, Chengdong Xu, Jingqi Liu, Xueying Ma, Zhiwen Zheng, Xiaofei Zhang,

-
- Bincheng Wang, Nichen Yang, Jie Wu, Lihua Tian, Chen Li, and Xuming Hu. Androtmem: From interaction trajectories to anchored memory in long-horizon gui agents, 2026. URL <https://arxiv.org/abs/2603.18429>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Yifan Sui, Xin Huang, Hongbing Li, Fang Xu, Jiahe Lv, Haolong Yan, Yeqing Shen, Litao Liu, Zhimin Fan, Ziyang Meng, Jia Wang, Junbo Qi, Kaijun Tan, Zheng Ge, Xiangyu Zhang, Daxin Jiang, and Osamu Yoshie. Androiddaily: A verifiable benchmark for mobile gui agents on real-world closed-source applications, 2026. URL <https://arxiv.org/abs/2605.27761>.
- Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Weizheng Wang, Hongzhang Huang, Jun Zhou, Jiachen Song, Shaoli Yu, Jinqi Wang, Zihang Zhou, Hongyi Zhou, Yuting Lv, Jinyang Li, Jiashuo Liu, Ruoyu Chen, Chunwei Liu, Guoliang Li, Jihua Kang, and Fan Wu. Workspace-bench 1.0: Benchmarking ai agents on workspace tasks with large-scale file dependencies, 2026. URL <https://arxiv.org/abs/2605.03596>.
- Harness Bench Team. Harness bench: Measuring harness effects in realistic agent workflows, 2026. URL <https://arxiv.org/abs/2605.27922>. 106 sandboxed offline agent tasks across 8 categories.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=OJd3ayDDoF>.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents, 2025. URL <https://arxiv.org/abs/2504.12516>.
- Dingbang Wu, Rui Hao, Haiyang Wang, Shuzhe Wu, Han Xiao, Zhenghong Li, Bojiang Zhou, Zheng Ju, Zichen Liu, Lue Fan, and Zhaoxiang Zhang. Mobilegym: A verifiable and highly parallel simulation platform for mobile gui agent research, 2026a. URL <https://arxiv.org/abs/2605.26114>.
- Qinzhao Wu, Zhizhuo Yang, Hanhao Li, Pengzhi Gao, Wei Liu, and Jian Luan. Mobilebench-ol: A comprehensive chinese benchmark for evaluating mobile gui agents in real-world environment. *arXiv preprint arXiv:2601.20335*, 2026b.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024.
- Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, et al. Mobile-agent-v3. 5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*, 2026.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2405.15793>.
- John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMillin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. Program-bench: Can language models rebuild programs from scratch?, 2026. URL <https://arxiv.org/abs/2605.03546>.

-
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 20744–20757. Curran Associates, Inc., 2022a. doi: 10.52202/068431-1508. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022b.
- Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, Weiming Wu, Jiayang Sun, Jiamin Song, Kaiqian Cui, Bowen Wang, Haoyuan Wu, Yitong Li, Dunjie Lu, Haikong Lu, Qi Zhen, Xinyuan Wang, Jiaqi Deng, Yuhao Yang, Cheng Chen, Boyuan Zheng, Alex Su, Xiao Yu, Hao Zou, Saaket Agashe, Xing Han Lu, Manpreet Kaur, Zhengyang Qi, Vincent Sunn Chen, Frederic Sala, Dayiheng Liu, Junyang Lin, Zhou Yu, Yu Su, Siva Reddy, Xin Eric Wang, Peng Qi, Tianbao Xie, and Tao Yu. Osworld2.0: Benchmarking computer use agents on long-horizon real-world tasks, 2026. URL <https://arxiv.org/abs/2606.29537>.
- Yujie Zheng, Zikang Liu, Xin Zhao, and Ji-Rong Wen. A task-state representation for long-horizon mobile gui agents, 2026. URL <https://arxiv.org/abs/2607.00502>.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- Xurui Zhou, Gongwei Chen, Yuquan Xie, Zaijing Li, Kaiwen Zhou, Shuai Wang, Shuo Yang, Zhuotao Tian, and Rui Shao. Hiconagent: History context-aware policy optimization for gui agents, 2026. URL <https://arxiv.org/abs/2512.01763>.